

Bensu Evaline Efe

Professor Dawn Harrison

CSC 121 – Python Programming

07 May 2026

Final Project - Learning Log Final Project Documentation

Section 1 - Project Overview

The Learning Log application is a web-based Django project developed by following the Learning Log project in Python Crash Course, 3rd Edition by Eric Matthes. The purpose of this application is to allow users to create personal learning topics and store entries under each topic. Users can register for an account, log in securely, create topics, add learning entries, and edit existing entries. The application uses Django's authentication system for secure user management and Bootstrap for styling and layout design.

The project demonstrates the use of Django models, views, templates, forms, authentication, authorization, URL routing, and database relationships. Security measures were implemented so that each user can only access and edit their own topics and entries.

Section 2 - How to Use the Application

1. Open the application in a browser using the local server address:

`http://127.0.0.1:8000/`

2. On the home page, click **Register** to create a new account.
3. Create a username and password to register.
4. After registration, the user is automatically logged into the application.
5. Click **Topics** in the navigation bar to view all personal learning topics.
6. Click **New Topic** to create a new topic.
7. Enter the topic title and save it.
8. Click on any topic to open its detail page.
9. Add a new entry under the selected topic.
10. Edit an existing entry by clicking the **Edit Entry** button.
11. Use the navigation bar to log out when finished.

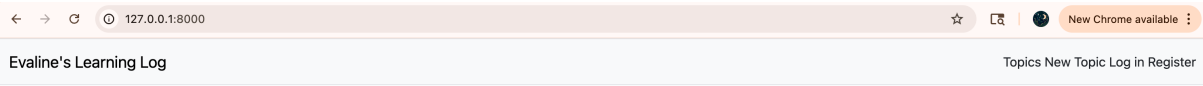
Security Note:

Each user can only view and edit their own topics and entries.

Section 3 - Screenshots

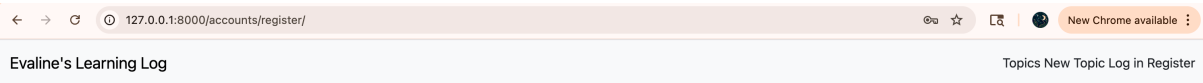
Screenshot 1: Home Page

The application home page where users can navigate, log in, or register.



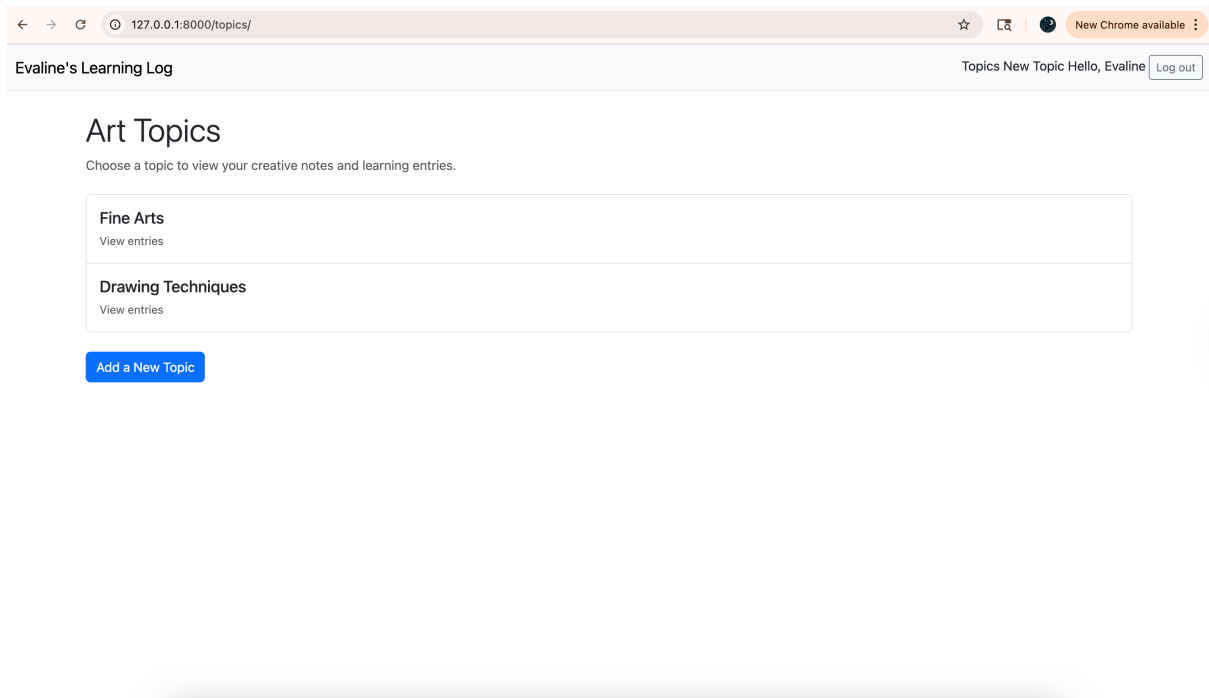
Screenshot 2: Registration Page

The registration page where new users create an account.



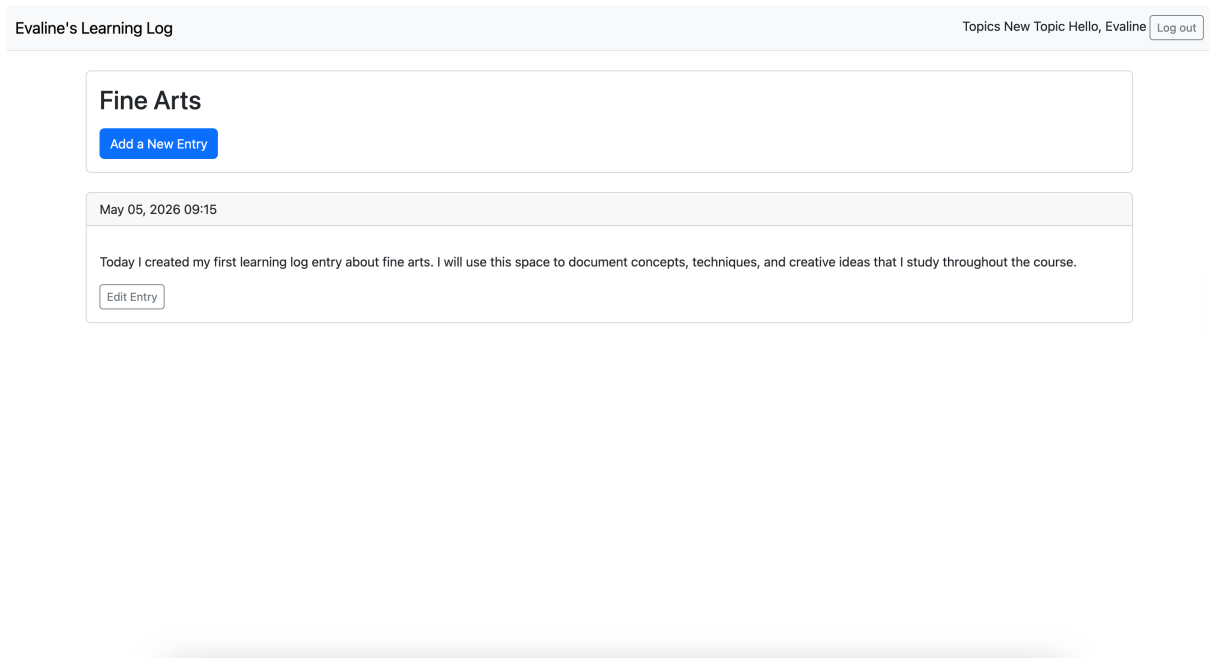
Screenshot 3: Topics Page

The topics page displaying all topics created by the logged-in user.



Screenshots 4: Topic Detail Page

The topic detail page showing all entries related to a selected topic.



Drawing Techniques

Add a New Entry

May 05, 2026 10:00

Today I continued working on my self-portrait and focused on building a stronger composition while incorporating personal symbolism into the piece. I spent time refining facial proportions, improving value contrast, and studying how light and shadow shape the structure of the face. I also experimented with expressive details and symbolic elements to make the artwork feel more connected to my identity, inner world, and artistic vision. This process helped me understand how technical accuracy and emotional storytelling can work together in portrait drawing.

I also learned how revision helps strengthen the final artwork.

Edit Entry

Screenshots 5: New Entry Page

The page where users can add a new entry under a selected topic.

Fine Arts

Add a new entry:

Add Entry

Drawing Techniques

Add a new entry:

Add Entry

Screenshots 6: Edit Entry Page

The page where users can edit an existing learning entry.

Edit Entry

Today I created my first learning log entry about fine arts. I will use this space to document concepts, techniques, and creative ideas that I study throughout the course.

Save Changes

Edit Entry

Today I continued working on my self-portrait and focused on building a stronger composition while incorporating personal symbolism into the piece. I spent time refining facial proportions, improving value contrast, and studying how light and shadow shape the structure of the face. I also experimented with expressive details and symbolic elements to make the artwork feel more connected to my identity, inner world, and artistic vision. This process helped me understand how technical accuracy and emotional storytelling can work together in portrait drawing.

I also learned how revision helps strengthen the final artwork.

Save Changes

Section 4 — Resources Used

Matthes, Eric. *Python Crash Course, 3rd Edition: A Hands-On, Project-Based Introduction to Programming*. No Starch Press.

(Primary textbook used for building the Learning Log application.)

[Django Documentation](#)

Used for reference on models, views, authentication, and URL routing.

[Bootstrap Documentation](#)

Used for styling and layout improvements.

[GitHub](#)

Used for project version control and project submission.

Section 5 — Code Printout

The following code files were used to build the Learning Log application. These files contain the application logic, models, forms, URL routing, and user authentication features.

File: learning_logs/models.py

```
from django.db import models
from django.contrib.auth.models import User

# Topic model stores subjects the user wants to learn
class Topic(models.Model):
    text = models.CharField(max_length=200)
    date_added = models.DateTimeField(auto_now_add=True)
    owner = models.ForeignKey(User, on_delete=models.CASCADE, default=1)

    def __str__(self):
        return self.text

# Entry model stores notes under a topic
class Entry(models.Model):
    topic = models.ForeignKey(Topic, on_delete=models.CASCADE)
    text = models.TextField()
    date_added = models.DateTimeField(auto_now_add=True)

    class Meta:
        verbose_name_plural = 'entries'

    def __str__(self):
        return self.text[:50]
```

File: learning_logs/views.py

```
from django.shortcuts import render, redirect
from django.http import Http404
from django.contrib.auth.decorators import login_required
from .models import Topic, Entry
from .forms import TopicForm, EntryForm

# Home page
def index(request):
    return render(request, 'learning_logs/index.html')

# Show all topics that belong to the logged-in user
@login_required
def topics(request):
    topics = Topic.objects.filter(owner=request.user).order_by('date_added')
    context = {'topics': topics}
    return render(request, 'learning_logs/topics.html', context)

# Show a single topic and all its entries
@login_required
def topic(request, topic_id):
```

```

topic = Topic.objects.get(id=topic_id)

# Prevent users from viewing topics they do not own
if topic.owner != request.user:
    raise Http404

entries = topic.entry_set.order_by('-date_added')
context = {'topic': topic, 'entries': entries}
return render(request, 'learning_logs/topic.html', context)

# Allow users to add a new topic
@login_required
def new_topic(request):
    if request.method != 'POST':
        form = TopicForm()
    else:
        form = TopicForm(data=request.POST)
        if form.is_valid():
            new_topic = form.save(commit=False)
            new_topic.owner = request.user
            new_topic.save()
            return redirect('learning_logs:topics')

    context = {'form': form}
    return render(request, 'learning_logs/new_topic.html', context)

# Allow users to add a new entry to a topic
@login_required
def new_entry(request, topic_id):
    topic = Topic.objects.get(id=topic_id)

    # Prevent users from adding entries to topics they do not own
    if topic.owner != request.user:
        raise Http404

    if request.method != 'POST':
        form = EntryForm()
    else:
        form = EntryForm(data=request.POST)
        if form.is_valid():
            new_entry = form.save(commit=False)
            new_entry.topic = topic
            new_entry.save()
            return redirect('learning_logs:topic', topic_id=topic_id)

    context = {'topic': topic, 'form': form}
    return render(request, 'learning_logs/new_entry.html', context)

# Allow users to edit an existing entry

```

```

@login_required
def edit_entry(request, entry_id):
    entry = Entry.objects.get(id=entry_id)
    topic = entry.topic

    # Prevent users from editing entries that belong to other users
    if topic.owner != request.user:
        raise Http404

    if request.method != 'POST':
        form = EntryForm(instance=entry)
    else:
        form = EntryForm(instance=entry, data=request.POST)
        if form.is_valid():
            form.save()
            return redirect('learning_logs:topic', topic_id=topic.id)

    context = {'entry': entry, 'topic': topic, 'form': form}
    return render(request, 'learning_logs/edit_entry.html', context)

```

File: learning_logs/forms.py

```

from django import forms
from .models import Topic, Entry

# Form for creating a new topic
class TopicForm(forms.ModelForm):
    class Meta:
        model = Topic
        fields = ['text']
        labels = {'text': ''}

# Form for creating a new entry under a topic
class EntryForm(forms.ModelForm):
    class Meta:
        model = Entry
        fields = ['text']
        labels = {'text': ''}
        widgets = {'text': forms.Textarea(attrs={'cols': 80})}

```

File: learning_logs/urls.py

```

from django.urls import path
from . import views

# Namespace for the learning_logs app
app_name = 'learning_logs'

```

```
# URL patterns for the learning log application
urlpatterns = [
    path('', views.index, name='index'), # Home page
    path('topics/', views.topics, name='topics'), # All topics
    path('topics/<int:topic_id>/', views.topic, name='topic'), # Single topic
    detail
    path('new_topic/', views.new_topic, name='new_topic'), # Add a new topic
    path('new_entry/<int:topic_id>/', views.new_entry, name='new_entry'), # Add a
    new entry
    path('edit_entry/<int:entry_id>/', views.edit_entry, name='edit_entry'), #
    Edit an entry
]
```

File: accounts/views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth import login
from django.contrib.auth.forms import UserCreationForm

# Register a new user and log them in automatically
def register(request):
    if request.method != 'POST':
        # Display a blank registration form
        form = UserCreationForm()
    else:
        # Process the completed registration form
        form = UserCreationForm(data=request.POST)

        if form.is_valid():
            new_user = form.save()
            login(request, new_user)
            return redirect('learning_logs:index')

    context = {'form': form}
    return render(request, 'registration/register.html', context)
```

File: accounts/urls.py

```
from django.urls import path, include
from . import views

# Namespace for the accounts app
app_name = 'accounts'

# URL patterns for authentication and registration
urlpatterns = [
    path('', include('django.contrib.auth.urls')), # Login, logout, password
    management
    path('register/', views.register, name='register'), # User registration page
]
```

```
]
```

File: ll_project/urls.py

```
"""
URL configuration for ll_project project.

The `urlpatterns` list routes URLs to views. For more information please see:
    https://docs.djangoproject.com/en/6.0/topics/http/urls/
Examples:
Function views
    1. Add an import:  from my_app import views
    2. Add a URL to urlpatterns:  path('', views.home, name='home')
Class-based views
    1. Add an import:  from other_app.views import Home
    2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
Including another URLconf
    1. Import the include() function: from django.urls import include, path
    2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
"""
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('accounts/', include('accounts.urls')),
    path('', include('learning_logs.urls')),
]
```

File: ll_project/settings.py

```
"""
Django settings for ll_project project.

Generated by 'django-admin startproject' using Django 6.0.4.

For more information on this file, see
https://docs.djangoproject.com/en/6.0/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/6.0/ref/settings/
"""

from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent
```

```
# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-19$sp^t$lj#^-v5##8u+4n%#x+6h$(=2)cn&yblo_z2119tccp'

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = []

# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'learning_logs',
    'accounts',
    'django_bootstrap5',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'll_project.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

```

]

WSGI_APPLICATION = 'll_project.wsgi.application'

# Database
# https://docs.djangoproject.com/en/6.0/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

# Password validation
# https://docs.djangoproject.com/en/6.0/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

# Internationalization
# https://docs.djangoproject.com/en/6.0/topics/i18n/

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True

# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/6.0/howto/static-files/

STATIC_URL = 'static/'

```

```
# Redirect users to the home page after login
LOGIN_REDIRECT_URL = 'learning_logs:index'

# Redirect users to the home page after logout
LOGOUT_REDIRECT_URL = 'learning_logs:index'
```